

1. Contexte et continuité avec la phase 2

La phase 2 a transformé l'application « MTL Vélo » en une vraie application client-serveur: la frontale s'est enrichie de cartes interactives (Leaflet) et de graphiques (Chart.js), une dorsale Node.js + Express expose une première API REST, et une base SQLite stocke les données de comptage. La phase 3 fait évoluer ce socle sur trois axes: compléter l'API REST (pagination, filtres, requêtes géospatiales), introduire l'authentification et la sécurisation par jetons, et intégrer un assistant conversationnel à la frontale.

Réutilisation et évolution du travail de la phase 2: la frontale conserve sa structure et ses cartes; la dorsale et la base SQLite sont conservées et étendues. Les routes embryonnaires de la phase 2 sont enrichies (pagination, filtres délégués à la base, agrégation par intervalle), et une nouvelle table utilisateurs rejoint le schéma. Les routes d'écriture sur les points d'intérêt sont protégées par jeton. Le journal de démarche est poursuivi tout au long de la phase 3.

Les deux parcours restent valorisés à égalité, y compris pour la fonctionnalité conversationnelle:

- **Parcours avec IA:** le Vélobot est implémenté en intégrant un grand modèle de langage (LLM) externe - appel d'API, prompt système, ancrage sur les données via RAG ou tool-calling, garde-fous, observabilité. La phase 3 introduit ainsi la notion d'IA comme composant applicatif et non plus seulement comme outil de développement.
- **Parcours sans IA:** le Vélobot est implémenté de manière entièrement déterministe - détection d'intention par mots-clés ou expressions régulières, requêtes SQL sur la base, composition de la réponse par gabarits en français. Cette approche est rigoureuse, robuste et professionnellement valable: aucune dépendance externe, aucun coût récurrent, comportement parfaitement reproductible. C'est l'occasion d'apprendre à concevoir une interface conversationnelle sans recourir à un LLM.

Aucun parcours n'est privilégié: les deux permettent d'atteindre la note maximale (100 / 100) et les deux livrent une fonctionnalité identique du point de vue de l'expérience. L'équipe peut conserver le parcours déclaré à la phase 2 ou en changer pour la phase 3; le nouveau choix est consigné dans **DEMARCHE.md** dès la première séance de la phase.

2. Objectifs pédagogiques de la phase 3

Objectifs communs aux deux parcours:

- **Concevoir une API REST complète** - pagination, filtres délégués à la base, requêtes géospatiales et auto-documentation à la racine.
- **Mettre en place une authentification** locale ou OAuth, avec hachage des mots de passe (bcrypt ou argon2) et émission/vérification de jetons (JWT).
- **Sécuriser les routes d'écriture** (POST, PUT, DELETE) par jeton et gérer les secrets en variables d'environnement.
- **Intégrer une fonctionnalité conversationnelle** « Vélobot » qui répond à des questions en langage naturel sur le réseau cyclable, **ancrée sur les données de la base.**

- **Tenir le journal de démarche** amorcé en phase 1 et étendu en phase 2, avec une rubrique dédiée à la fonctionnalité conversationnelle.

Objectifs spécifiques au parcours avec IA:

- **Intégrer un LLM** comme composant applicatif: choix d'un fournisseur, gestion des secrets côté serveur uniquement, conception d'un prompt système, ancrage des réponses (RAG ou tool-calling), gestion des coûts et limitation de débit.
- **Concevoir une UX honnête pour l'IA** - indication explicite que les réponses sont générées, possibilité de signaler une mauvaise réponse, transparence sur les sources.

Objectifs spécifiques au parcours sans IA:

- **Concevoir une interface conversationnelle déterministe** - détection d'intention robuste (mots-clés, expressions régulières), gabarits de réponses en français, traitement des cas limites (intention non reconnue, données absentes).
- **Documenter les compromis** entre une approche déterministe (prévisible, sans coût, limitée en flexibilité) et une approche LLM (flexible mais avec coût, latence et risque d'hallucinations).

3. Tâches à réaliser

T1: API REST complète et auto-documentée

T1.1: Une requête `GET /gti525/v1/` retourne un JSON listant tous les points de terminaison disponibles avec une brève description et le verbe HTTP attendu.

T1.2: Toutes les données (compteurs, pistes, points d'intérêt, passages) sont accessibles via la base. Les opérations de filtrage et de pagination doivent être **déléguées à la base** (pas de filtrage applicatif sur des collections complètes chargées en mémoire).

T2: Ressources compteurs et points d'intérêt

T2.1: `GET /compteurs` - liste paginée avec paramètres `limite`, `page`, `implantation` (année minimale), `nom` (recherche textuelle), `arrondissement`. Réponse: { `donnees`, `total`, `page`, `limite` }.

T2.2: `GET /compteurs/:id` - informations d'un compteur (sans les passages).

T2.3: `GET /compteurs/:id/passages` avec paramètres `debut`, `fin` (YYYY-MM-DD), `intervalle` (`jour`, `semaine`, `mois`) - déjà esquissé en phase 2, à étoffer ici.

T2.4: `GET /pointsdinteret` - liste paginée avec filtres par `type`, `arrondissement`, `nom`.

T2.5: Opérations `POST`, `PUT` et `DELETE` sur `/pointsdinteret`, **protégées par jeton** (voir T4). Toute requête sans jeton ou avec un jeton invalide retourne 401.

T3: Ressource pistes cyclables et requête géospatiale

T3.1: `GET /pistes` - retourne une `FeatureCollection` GeoJSON construite à partir de la base. Paramètres `arrondissement`, `categorie`, `populaireDebut` et `populaireFin` (active la logique « pistes populaires »).

T3.2: Logique des pistes populaires: calculer les trois arrondissements avec le ratio $\Sigma \text{ passages} / N \text{ compteurs}$ le plus élevé sur la période demandée, puis **retourner toutes les pistes traversant ces arrondissements** (requête géospatiale dans la base ou calcul applicatif documenté).

T4: Authentification et sécurité

T4.1: Mettre en place une table `utilisateurs` (courriel unique, mot de passe haché, métadonnées) et deux routes: `POST /auth/inscription` et `POST /auth/connexion`. À la connexion, le serveur émet un jeton (JWT recommandé) avec une durée de vie limitée (24 h par exemple).

T4.2: Hachage des mots de passe avec un algorithme reconnu (bcrypt avec au moins 10 rounds, ou argon2). **Aucun mot de passe en clair** dans la base, dans les logs ou dans les réponses HTTP.

T4.3: Les routes `POST`, `PUT` et `DELETE` sur `/pointsdinteret` requièrent un jeton valide dans l'en-tête `Authorization: Bearer ....` Vérification effective de la signature (et non simple décodage).

T4.4: Gestion des secrets: clés JWT, clés d'API externes et tout autre secret sont stockés en variables d'environnement (ou dans un fichier `.env` exclu du dépôt). Aucun secret ne doit être commité dans le dépôt - y compris dans l'historique. Un fichier `.env-example` sans valeurs sensibles est attendu pour documenter les variables nécessaires.

T5: Évolutions de la frontale

T5.1: Pages d'inscription et de connexion (authentification locale ou OAuth au choix de l'équipe). L'interface s'adapte à l'état de connexion: barre de navigation différente, boutons Modifier/Supprimer visibles seulement si authentifié.

T5.2: Vue ou modale d'ajout, de modification et de suppression d'un point d'intérêt, avec un formulaire dont les champs s'adaptent au type sélectionné.

T5.3: La recherche de compteurs et le filtrage des points d'intérêt passent désormais par l'API et utilisent la pagination retournée (Précédent / Suivant, Page X sur Y).

T5.4: Les champs de date de la vue « Réseau cyclable » déclenchent l'affichage des pistes les plus populaires via `/pistes?populaireDebut=...&populaireFin=....`

T6: Fonctionnalité conversationnelle « Vélobot »

Cette tâche est la nouveauté principale de la phase 3 et l'occasion de réfléchir aux choix d'architecture conversationnelle. L'utilisateur peut poser des questions en langage naturel sur le réseau cyclable et reçoit une réponse ancrée sur les données de l'application.

T6.1: Ajouter une vue « Assistant » accessible depuis le menu principal, avec une zone de conversation et un champ de saisie limité à 1 000 caractères.

T6.2: Implémenter la route `POST /gti525/v1/assistant` qui reçoit une question, consulte la base pour rassembler les données pertinentes, compose une réponse et la retourne. L'implémentation est libre selon le parcours déclaré (voir T6.5).

T6.3: L'assistant doit répondre correctement à au moins **trois familles de questions** parmi: statistiques de passages sur une période, recherche d'un point d'intérêt par proximité, informations sur les pistes (longueur, catégorie) dans un arrondissement, identification de la piste la plus achalandée, comparaison entre deux périodes ou deux arrondissements.

T6.4: Ancrage et honnêteté: l'assistant ne doit pas inventer de chiffres. Si la donnée n'est pas disponible, il doit le dire explicitement. L'interface indique clairement que les réponses sont générées et permet à l'utilisateur de signaler une mauvaise réponse (un simple bouton qui consigne l'événement dans un journal serveur est suffisant).

T6.5: Implémentation selon le parcours

- **Parcours avec IA:** appel à un LLM externe (Anthropic, OpenAI, Mistral, etc.) côté serveur uniquement. **La clé d'API ne doit jamais être exposée à la frontale.** Le serveur compose un prompt système clair, fournit le contexte issu de la base (RAG simple ou tool-calling si supporté), valide la longueur de la question (1 000 caractères max), applique une limitation de débit simple par adresse IP, et journalise les appels (horodatage, longueur de la question, temps de réponse, présence d'erreur - sans données personnelles).
- **Parcours sans IA:** implémentation entièrement déterministe. Détection d'intention par mots-clés ou expressions régulières, requêtes SQL paramétrées sur la base pour rassembler les données, composition de la réponse par gabarits en français. Cas de repli explicite quand l'intention n'est pas reconnue (« Je ne peux pas répondre à cette question avec les données disponibles. »). Mêmes garde-fous: validation de la longueur de la question, limitation de débit, journalisation simple.

T7: Démarche réflexive et journal d'équipe

Le journal poursuivi depuis la phase 1 reçoit une nouvelle rubrique consacrée à l'assistant conversationnel - c'est l'occasion de tracer une décision d'architecture importante (intégration d'un LLM ou conception déterministe) et ses conséquences techniques.

T7: Tâches communes aux deux parcours

- **T7.1:** Si l'équipe change de parcours pour la phase 3, mettre à jour **DEMARCHE.md** avec une brève justification.
- **T7.2:** Tenir le journal à jour avec au moins **deux entrées critiques par membre** sur des décisions de la phase 3 (par exemple : choix de l'authentification locale ou OAuth, conception de l'assistant, stratégie d'ancrage des réponses, choix d'une bibliothèque JWT).
- **T7.3:** Documenter dans le journal le **design de l'assistant**: intentions reconnues, format de réponse, gestion des cas non reconnus, exemples de questions-réponses traitées correctement et incorrectement.

T7: Parcours avec IA

- **T7.A.1:** Étoffer **PROMPTS.md** avec une rubrique « **Fonctionnalité conversationnelle** » en plus des rubriques Frontale, Dorsale et Revue critique. Chaque entrée suit la structure définie à **T3.A.1 (phase 1)** : prompt, outil et modèle, sortie obtenue, modifications humaines, et justification du jugement critique posé sur la sortie. Pour la fonctionnalité conversationnelle, consigner également le prompt système final, ses itérations principales (avant / après) et les cas où l'assistant a échoué (réponses inventées, refus inappropriés, etc.) avec les correctifs apportés.
- **T7.A.2:** Tenir la liste des hallucinations rencontrées dans le développement, étendue aux hallucinations du Vélobot lui-même (au moins deux exemples documentés).

T7: Parcours sans IA

- **T7.B.1:** Étoffer **DEMARCHE.md** avec une rubrique « **Fonctionnalité conversationnelle** »: architecture de détection d'intention choisie (table de mots-clés, regex, ou autre), stratégie de gabarits, gestion des cas non reconnus, exemples concrets.
- **T7.B.2:** Documenter une **réflexion comparative**: avantages de l'approche déterministe (prévisibilité, absence de coût, robustesse), limites (couverture, flexibilité), et conditions sous lesquelles l'équipe pourrait privilégier un LLM dans un projet futur.

Important: le choix du parcours peut être révisé en cours de phase à condition d'être documenté dans **DEMARCHE.md**. Une équipe ne peut pas changer de parcours après la démonstration. La transparence est elle-même évaluée - voir Section 7.

4. Livrables

À la fin des quatre séances, chaque équipe remet:

- **Le code source complet** sur le dépôt Git de l'équipe (le même qu'aux phases 1 et 2), avec un fichier **README.md** décrivant l'installation, les variables d'environnement requises (avec un `.env.example` sans valeurs sensibles) et le démarrage.
- **Le journal de démarche** à la racine du dépôt: **DEMARCHE.md** pour les deux parcours, accompagné de **PROMPTS.md** pour le parcours avec IA. Les deux fichiers conservent et étendent le contenu des phases précédentes.
- **Un rapport** PDF de 5 pages maximum (hors page de titre, table des matières et références).
- **Une démonstration** en direct de 15 à 20 minutes lors de la dernière séance, incluant: un aperçu de l'architecture globale, deux ou trois requêtes API en direct, l'authentification et une

route protégée, et le Vélobot sur trois questions différentes (dont une que le chargé de laboratoire posera lui-même).

Procédure de soumission - gel de la version évaluée

Comme aux phases précédentes, la version évaluée est figée à l'aide d'un **tag Git annoté** pour permettre à l'équipe de continuer à travailler librement sur le dépôt après la date limite.

Au moment de la soumission, et au plus tard à la date limite, chaque équipe:

- crée un tag Git annoté nommé **phase3-submission** sur le commit à évaluer:
 - `git tag -a phase3-submission -m "Soumission phase 3"`
- pousse le tag vers le dépôt distant:
 - `git push origin phase3-submission`
- récupère le SHA complet du commit ciblé:
 - `git rev-parse phase3-submission`
- dépose sur Moodle, dans le formulaire de remise: le lien du dépôt, le nom du tag, le SHA complet du commit, ainsi que le rapport PDF.

Évaluation: le chargé de laboratoire évalue exclusivement la version du code identifiée par le tag. Toute divergence ultérieure entre le tag (côté dépôt) et le SHA déposé sur Moodle est traitée comme une tentative de modification après la date limite et entraîne la pénalité P5.

5. Contenu attendu du rapport

Le rapport est limité à cinq pages et doit répondre aux questions suivantes. La rubrique R3 a deux variantes selon le parcours déclaré dans **DEMARCHE.md** - choisir et traiter celle qui s'applique.

R1: Architecture globale finale (frontale, dorsale, base, intégration Vélobot). Schéma à l'appui. Évolution depuis la phase 2.

R2: Sécurité: authentification choisie, gestion des jetons, mesures contre les injections, gestion des secrets. Discussion des menaces considérées.

R3: Démarche professionnelle et réflexion critique.

- **Parcours avec IA:** description de l'intégration LLM (modèle utilisé, prompt système commenté, mécanisme d'ancrage, garde-fous et limites), exemple concret d'un échec rencontré et de sa correction.
- **Parcours sans IA:** description de l'architecture conversationnelle déterministe (détection d'intention, gabarits, gestion des cas non reconnus), exemple concret d'un cas limite rencontré et de sa résolution, réflexion comparative avec l'approche LLM.

R4: Réflexion finale (1 page): compétences acquises sur les trois phases, apport de la démarche réflexive (avec ou sans IA) à l'apprentissage et à la préparation professionnelle, précautions prises pour conserver la maîtrise du code.

R5: Répartition des tâches dans l'équipe et brève introduction et conclusion.

6. Grille d'évaluation

La phase 3 vaut 100 points, répartis en trois volets : démonstration, code et rapport, selon la grille du chargé de laboratoire.

Deux parcours, évalués à égalité:

- **Avec IA:** usage d'un assistant pour générer, améliorer ou analyser du code et intégration d'un LLM dans le Vélobot.
- **Sans IA:** développement entièrement manuel et Vélobot implémenté de façon déterministe (intention par mots-clés ou regex, gabarits de réponses).

Les deux visent les mêmes compétences (pratique réflexive d'ingénierie, conception d'une interface conversationnelle ancrée sur les données) et permettent d'obtenir la note maximale.

Déclaration du parcours: inscrit dans **DEMARCHE.md** à la racine du dépôt dès la première séance de la phase. Le choix peut être révisé s'il est documenté; les critères C5 et R3 applicables sont alors ajustés.

6.1 Démonstration en direct (35 points)

Critère	Description	Points
D1 - API REST complète	Routes complètes, paginées et filtrées, racine /gti525/v1/ auto-documentée. Réponses correctes en direct.	6
D2 - Pistes populaires	La fonctionnalité de pistes populaires fonctionne pour une période choisie en direct.	4
D3 - Frontale enrichie	CRUD des points d'intérêt depuis l'interface, recherche par API, filtres opérationnels.	5
D4 - Authentification	Inscription / connexion fonctionnelles ; route protégée refuse les requêtes sans jeton (401).	5
D5 - Vélobot	Trois familles de questions répondues correctement et avec ancrage sur les données. Une question imprévue posée par le chargé de laboratoire est traitée raisonnablement (réponse correcte si la	10

	donnée est disponible, refus explicite sinon). Aucun chiffre fabriqué non détecté.	
D6 - Maîtrise du code	Lors des questions, chaque membre est capable d'expliquer le code qu'il a contribué (frontale, dorsale, sécurité, Vélobot) et de justifier les choix techniques (et, le cas échéant, ce qui a été produit avec l'IA et comment il l'a évalué).	5

6.2 Code et dépôt Git (35 points)

Critère	Description	Points
C1 - API et base	Routes propres et complètes, validation des entrées, requêtes paramétrées, schéma cohérent, filtrage et pagination délégués à la base. Stratégie géospatiale claire.	8
C2 - Authentification et secrets	Mots de passe hachés (bcrypt ou argon2), JWT signé et vérifié, secrets en variables d'environnement, .env-exemple présent, aucun secret commité (y compris dans l'historique).	8
C3 - Frontale finale	Modulaire, gestion des états de connexion, gestion d'erreurs UX (chargements, erreurs API).	5
C4 - Vélobot	Implémentation conforme au parcours déclaré (voir ci-dessous). Question utilisateur validée (longueur), limitation de débit en place, journalisation simple sans données personnelles.	4
C5 - Journal de démarche	Tenue d'un journal de démarche réflexive selon le parcours choisi (voir ci-dessous). Les deux options sont équivalentes en points et en exigence.	10

C4 - Vélobot, selon le parcours:

- **Parcours avec IA:** clé d'API du LLM uniquement côté serveur (jamais exposée à la frontale), prompt système clair, ancrage explicite (RAG ou tool-calling).
- **Parcours sans IA:** détection d'intention robuste (mots-clés/regex), gabarits de réponses en français, cas de repli explicite quand l'intention n'est pas reconnue, requêtes SQL paramétrées.

C5 - Pratique réflexive et traçabilité:

- **Parcours avec IA: PROMPTS.md** à jour avec rubriques distinctes Frontale / Dorsale / Sécurité / Fonctionnalité conversationnelle; au moins deux entrées critiques par membre sur la phase 3; au moins deux hallucinations documentées.
- **Parcours sans IA: DEMARCHE.md** à jour avec les mêmes rubriques; pour chaque tâche: problème, sources, alternatives et justification; au moins deux entrées par membre sur la phase 3; réflexion documentée sur le design de l'assistant déterministe.

Dans les deux cas, l'évaluation porte sur la qualité de la réflexion et la traçabilité des décisions.

6.3 Rapport (30 points)

Critère	Description	Points
R1 - Architecture globale	Schéma global, évolution depuis la phase 2, choix justifiés.	5
R2 - Sécurité	Authentification justifiée, gestion des jetons, mesures contre l'injection, gestion des secrets, menaces considérées.	5
R3 - Démarche professionnelle et réflexion critique	Section dédiée du rapport rendant compte du parcours choisi (voir ci-dessous). Mêmes attentes de profondeur dans les deux cas ; un constat superficiel est insuffisant.	12
R4 - Réflexion finale	1 page sur les compétences acquises, l'apport de la démarche réflexive, et les précautions prises pour conserver la maîtrise du code. Une réflexion qui se réduit à une liste d'outils ou de tâches est insuffisante.	5
R5 - Répartition et présentation	Rôles précis, introduction et conclusion soignées, qualité de la mise en page.	3

R3 - Démarche et réflexion critique:

- **Parcours avec IA:** intégration LLM (modèle, prompt système commenté, mécanisme d'ancrage, garde-fous et limites); exemple concret d'un échec et de sa correction. Une simple liste d'outils est insuffisante.
- **Parcours sans IA:** architecture conversationnelle déterministe (détection d'intention, gabarits, cas non reconnus); exemple concret d'un cas limite et de sa résolution ; réflexion comparative avec l'approche LLM. Une réponse vague est insuffisante.

6.4 Pénalités

- **P1 - Rapport mal rédigé:** qualité du français insuffisante (clarté, structure, syntaxe): **jusqu'à -10 points.**
- **P2 - Parcours avec IA:** code généré accepté tel quel, sans relecture ni adaptation, comme l'indiquent le dépôt et le journal de prompts: **-10 points.**
- **P3 - Parcours avec IA:** absence ou non-respect du journal **PROMPTS.md** malgré un usage manifeste de l'IA: **-10 points.**
- **P4 - Parcours sans IA:** absence ou non-respect du journal **DEMARCHE.md**, ou usage d'une IA non déclaré: **-10 points.**
- **P5 - Soumission non conforme:** tag `phase3-submission` absent du dépôt, pointant sur un commit différent du SHA déposé sur Moodle, ou créé après la date limite Moodle: **-15 points.**
- **P6 - Secret commité dans le dépôt:** clé d'API, jeton ou autre secret présent dans le code ou dans l'historique Git: **-15 points.**
- **P7 - Mots de passe non sécurisés:** stockage en clair ou hachage trivial (MD5, SHA-1 sans sel): **-10 points.**
- **P8 - Injection SQL:** requête SQL construite par concaténation de paramètres utilisateur: **-10 points.**
- **P9 - Clé d'API exposée à la frontale (parcours avec IA):** appel direct à un LLM depuis le code frontal, clé visible côté client: **-10 points.**
- **P10 - Vélobot fabrique des chiffres:** réponses contenant des chiffres ou entités inventés, non détectés par les garde-fous, observés en démonstration: **-5 points par cas, jusqu'à -15 points.**

Note: P3 et P4 sont symétriques selon le parcours choisi. P9 ne s'applique qu'au parcours avec IA. P6 à P8 et P10 portent sur la qualité technique nouvelle introduite à la phase 3 et s'appliquent aux deux parcours.

7. Politique d'utilisation de l'IA et choix du parcours

Choix du parcours: libre et à la discrétion de l'équipe. À déclarer dans **DEMARCHE.md** dès la première séance de la phase, avec possibilité de révision si documentée. Les deux parcours sont équivalents et permettent d'atteindre 100 / 100.

Parcours avec IA - usages autorisés: génération de code, explication, refactorisation, génération de tests, suggestions de design, débogage assisté; intégration d'un LLM comme composant applicatif dans le Vélobot.

Parcours sans IA - ressources autorisées: documentation officielle (Express, SQLite, JWT, bcrypt, MDN, etc.), manuels, articles techniques, communautés (Stack Overflow pour comprendre, pas pour copier sans relecture). Les échanges en équipe et avec le chargé de laboratoire sont encouragés.

Interdictions (tous): soumettre du code sans relecture humaine, un rapport généré sans révision, du contenu copié d'une autre équipe ou session, un faux journal (**PROMPTS.md** / **DEMARCHE.md**), ou des secrets commités dans le dépôt.

Transparence: toute utilisation de l'IA doit être documentée. Les incohérences entre les déclarations et le code ou les commits sont pénalisées, dans les deux parcours.

Plagiat: le règlement de l'ÉTS s'applique. Le code est comparé entre sessions. La réutilisation de code d'une autre équipe ou d'une session antérieure est interdite.